

OpenGL Project Documentation

Irimes David

January 12, 2025

Contents

1	Introduction	2
2	Camera System	2
3	3D Model Loading	2
4	Texture Mapping	3
5	Lighting System	4
6	Scene View Modes	5
7	Shadow Mapping	6
8	Dynamic Scene Components	7
9	Advanced Realism Features	7
10	Possible Enhancements	8
11	Bibliography	9

1 Introduction

This project is designed to demonstrate the construction of a small game engine using OpenGL. The engine incorporates key functionalities, enabling both interactive and automatic scene navigation, realistic rendering, and dynamic scene manipulation. This document details the features and underlying techniques implemented in the project.

The primary features include:

- Camera navigation and animation.
- Loading and rendering 3D models using the `tiny_obj_loader` library.
- Comprehensive texture mapping.
- Multiple types of dynamic light sources.
- Scene view modes and shadow mapping.
- Dynamic and interactive scene components.
- Realistic rendering with advanced effects like skyboxes, fog, reflections, and transparency.
- Performance optimizations for large scenes and complex effects.

2 Camera System

The camera system allows for navigation through the scene using user inputs and provides an animation mode for automated scene exploration.

- **Interactive Mode:** Users can navigate the scene using keyboard and mouse inputs, adjusting the position, orientation, and zoom level of the camera.
- **Animation Mode:** A pre-defined camera path traverses the scene, offering an automatic showcase of the environment.

3 3D Model Loading

The engine utilizes the `tiny_obj_loader` library for loading 3D models in the OBJ format. Only OBJ files are supported.

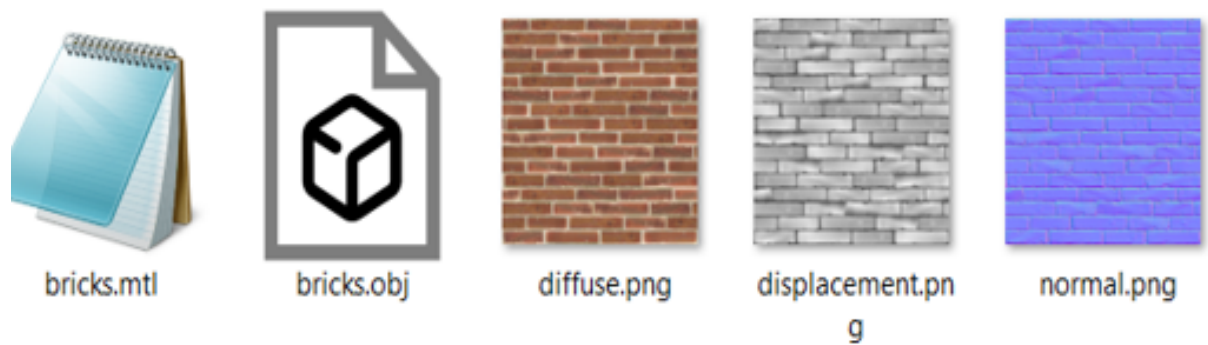
- **Material Support:** The associated material file (MTL) specifies textures and material properties.
- **Precomputed Vectors:** Tangent and Bitangent vectors are calculated on the CPU to enhance normal mapping and lighting effects.

4 Texture Mapping

Correct mapping of multiple texture types is implemented, supporting:

- Ambient
- Diffuse
- Specular
- Normal
- Displacement
- Emissive
- Metallic
- Alpha (transparency)

The textures are applied conditionally based on their presence in the material specification.



figureExamples of Texture Mapping Types Applied to Models

5 Lighting System

- **Light Types:** Point light, directional light, and spotlight.
- **Shading Model:** The Blinn-Phong model is used for realistic and efficient lighting calculations.
- **Dynamic Light Sources:** Lights can be moved, rotated, and adjusted for color and intensity. Light sources are visually represented by objects resembling light-bulbs.

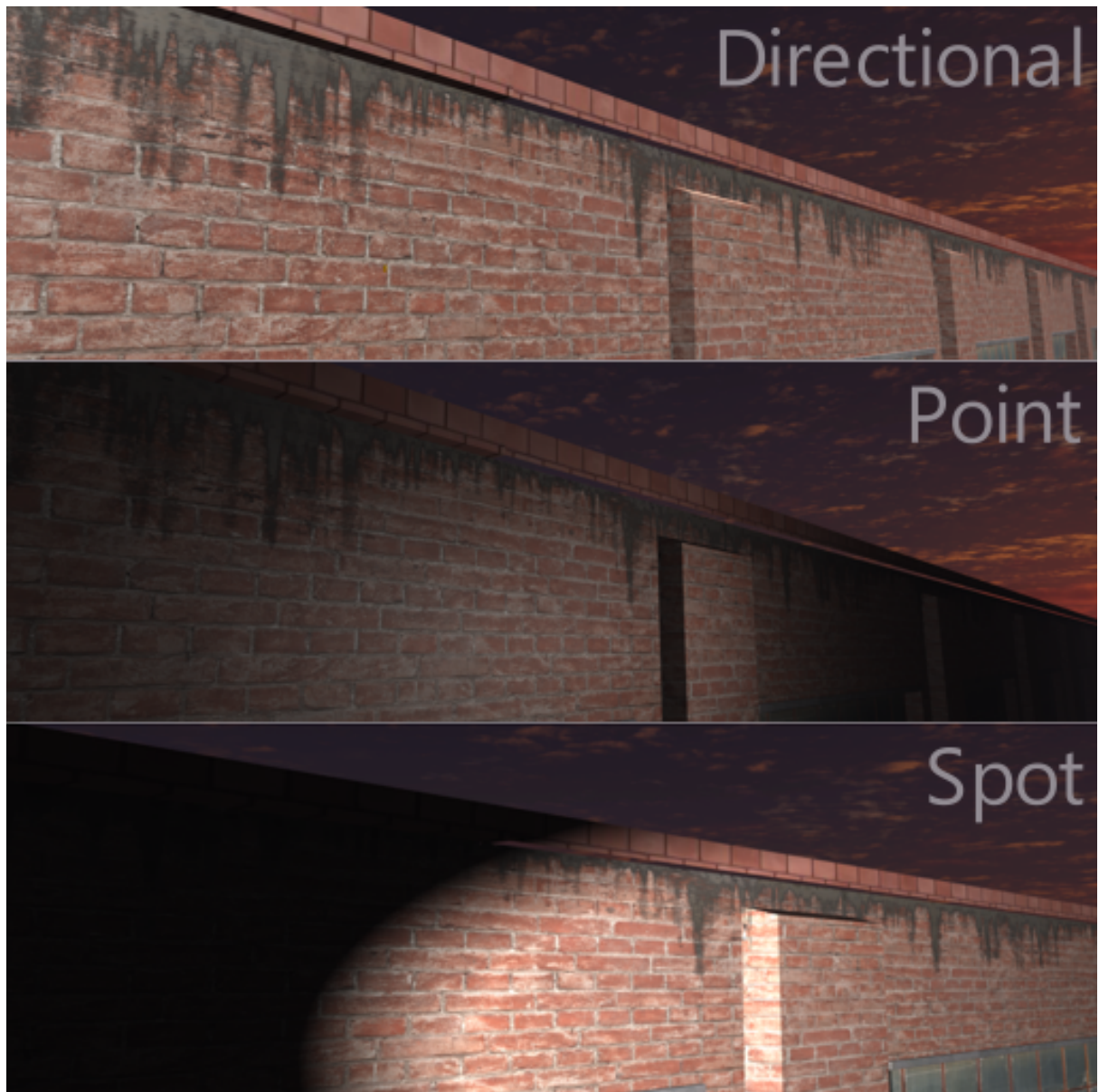


figure Different Light Types and Their Effects on a Model

6 Scene View Modes

The engine provides multiple scene visualization modes:

- **Normal Mode:** Includes textures, shadows, and lighting.
- **Stylized Mode:** Uses a stencil buffer to create an outline effect for a cartoonish appearance.
- **Wireframe Mode:** Displays the scene's underlying geometry by showing object triangles/faces.

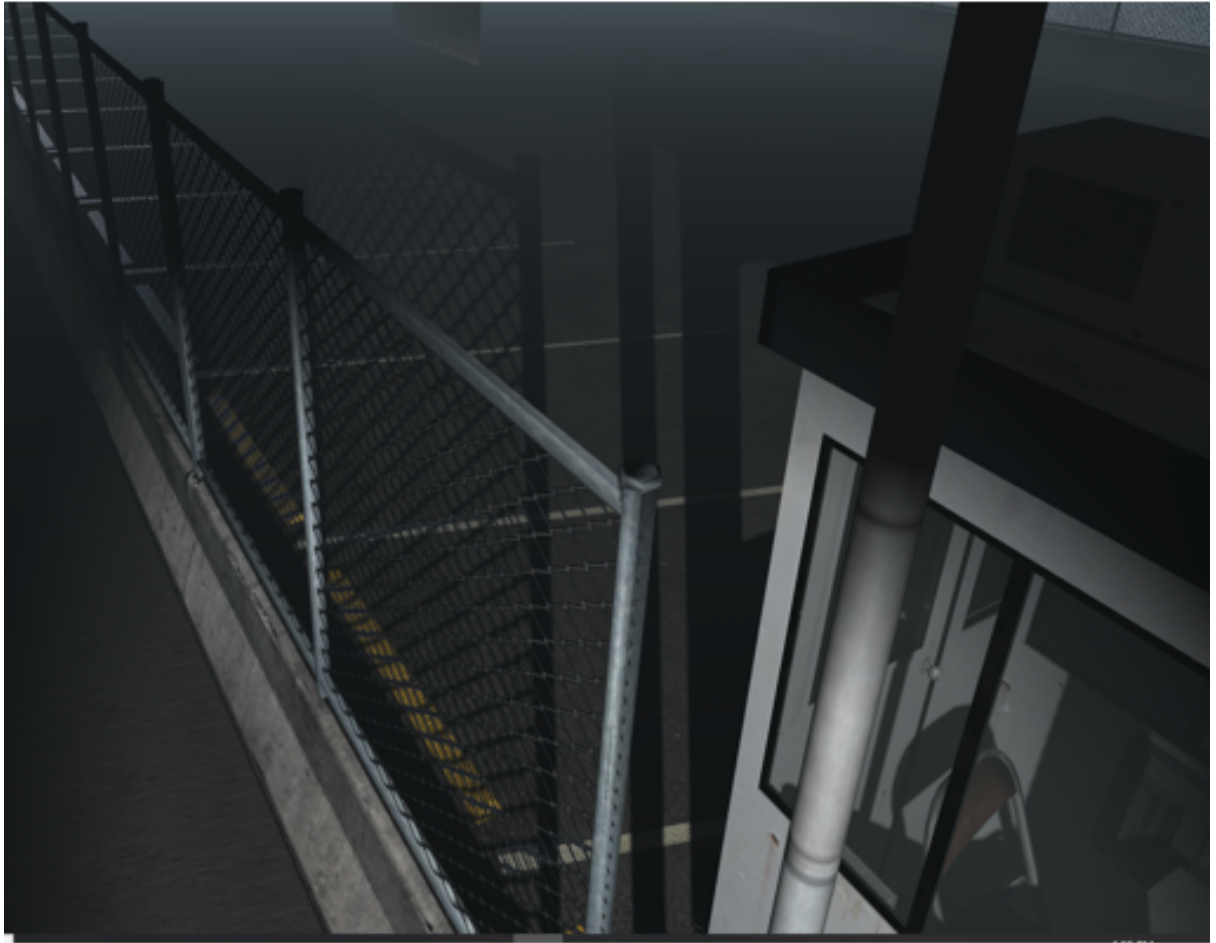


figureComparison of Scene View Modes

7 Shadow Mapping

Directional and spotlight shadows are implemented using shadow mapping techniques.

- **PCF (Percentage Closer Filtering):** Used for softening shadow edges, enhancing realism.
- **Depth Bias:** A small bias is applied to reduce shadow acne and improve rendering quality.



figureShadow Mapping Process

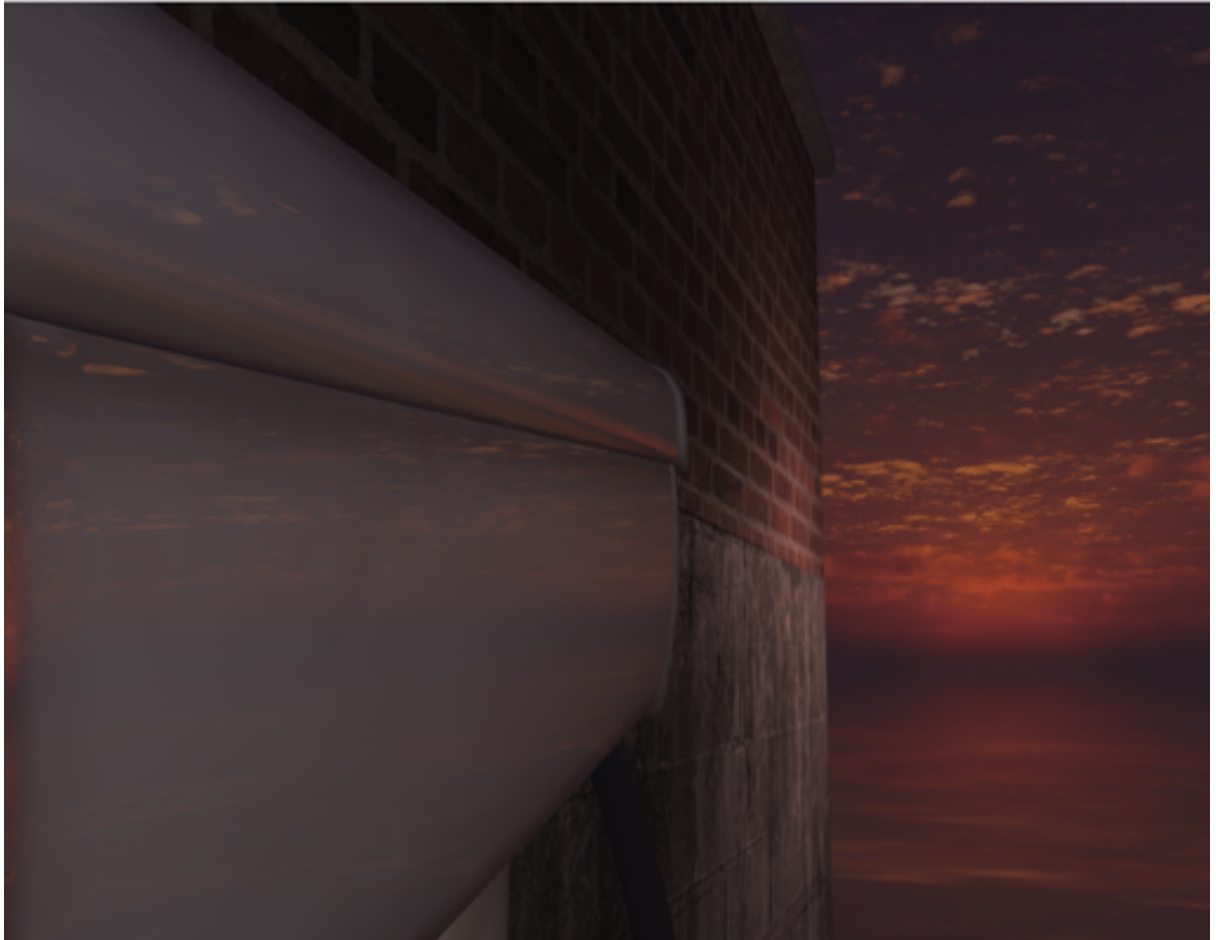
8 Dynamic Scene Components

- Light sources can be added or removed dynamically.
- Selection of light sources is highlighted using the outline functionality.
- Objects in the scene respond to dynamic lighting and can cast shadows.

9 Advanced Realism Features

- **Skyboxes:** Changeable skyboxes provide immersive backgrounds.
- **Fog:** Toggleable fog whose color adapts to the skybox for seamless blending.
- **Reflections:** Metallic texture maps dictate object reflectivity.

- **Transparency:** Alpha blending is used for semi-transparent objects, drawn at the end of the rendering cycle to handle depth buffer issues.
- **Post-Processing Effects:** Additional effects such as bloom and motion blur can enhance scene realism.



figureSkybox and Fog Integration with Reflections

10 Possible Enhancements

- Expand support for additional 3D model formats (e.g., FBX, GLTF).
- Implement object manipulation in real time, not only for light objects (translation, rotation, scale).
- Introduce a graphical user interface (GUI) for easier scene manipulation.

- Optimize performance using techniques such as frustum culling and level-of-detail (LOD).
- Add physics-based rendering (PBR) for even more realistic materials and lighting.

11 Bibliography

- OpenGL Tutorial Series by XYZ
- OGLDEV YouTube Channel
- Learn OpenGL Website